

TaskFlow – Trabajo final módulo 5

Eduardo Gomes

Objetivo del Proyecto. Desarrollar una aplicación web de gestión de tareas que permita crear, editar, eliminar, completar y filtrar tareas, incorporando Programación Orientada a Objetos, eventos del DOM, asincronía, consumo de API y persistencia local.

1. Demostración funcional

Funcionalidad	Cómo se prueba	Resultado esperado
Crear tarea	Escribir una descripción, opcionalmente una fecha, y pulsar “Agregar”.	La tarea aparece en pantalla y se guarda en localStorage.
Cambiar estado	Pulsar el botón check de una tarea.	La tarea cambia entre “Pendiente” y “Completada”.
Editar tarea	Pulsar el lápiz para editar y modificar descripción o fecha.	La tarea actualiza sus datos y se vuelve a renderizar.
Eliminar tarea	Pulsar papelerero para eliminar.	La tarea desaparece de la lista y de localStorage.
Filtrar tareas	Usar “Todas”, “Pendientes” y “Completadas”.	La interfaz muestra solo las tareas del filtro activo.
Persistencia local	Recargar la página después de crear tareas.	Las tareas siguen visibles porque fueron guardadas en localStorage.
GET a API	Pulsar “Cargar 5 tareas demo”.	Se consumen datos desde JSONPlaceholder y se agregan tareas demo a la app.
POST a API	Pulsar “Guardar una tarea demo (POST)”.	Se ejecuta una simulación de guardado y se muestra el id devuelto por la API.
Asincronía visual	Agregar una tarea con fecha límite.	Se observa retardo controlado con setTimeout y contador regresivo con setInterval.

2. Explicación breve del código

index.html: Contiene la estructura principal de la interfaz: formulario, lista de tareas, filtros, acciones de API y contenedor de notificaciones.

styles.css: Aplica la presentación visual del proyecto.

app.js: Incluye toda la lógica de la aplicación: clases, eventos, renderizado, asincronía, consumo de API y almacenamiento local.

Arquitectura lógica. El sistema se organiza alrededor de dos clases. **Tarea** modela una tarea individual con propiedades y métodos propios. **GestorTareas** centraliza el arreglo de tareas y encapsula operaciones como agregar, editar, eliminar, filtrar y persistir.

Conceptos de JavaScript aplicados. Se utilizaron clases, campos privados, arrow functions, template literals, destructuring, spread operator, addEventListener, manipulación del DOM, setTimeout, setInterval, fetch, async/await, try/catch y localStorage.

Documentación del Código: para que se entienda mejor qué hace cada parte.